What is Overfitting?

Overfitting is a machine learning problem where a model learns the training data **too well**, including its noise, random fluctuations, and irrelevant details. As a result, it performs **very well on the training data** but **poorly on new, unseen data**.

In other words:

- **Training accuracy:** Very high
- **Testing/Validation accuracy:** Low

The model memorizes the data instead of learning the underlying patterns.

Why Does Overfitting Occur?

Overfitting usually happens because:

1. The model is too complex.
2. The training dataset is too small.
3. The training data contains noise.
4. The model is trained for too many epochs.
5. Too many unnecessary features are included.

Characteristics of Overfitting

- Very low training error
- High testing error
- Poor generalization
- Large gap between training and validation accuracy

Overfitting is associated with **high variance**.

- **Underfitting:** High bias, low variance
- **Good fit:** Balanced bias and variance
- **Overfitting:** Low bias, high variance

Bias is the difference between the model's average prediction and the true value.

Variance measures how much a model's predictions change when it is trained on different datasets.

Due to overfitting

High training accuracy
Low validation accuracy
Validation loss increases while training loss decreases
Model performs poorly on unseen data

Techniques to Solve Overfitting

1. Increase Training Data

More diverse data helps the model learn general patterns instead of memorizing specific examples.

Example:

- Before: 500 images
- After: 50,000 images

This generally improves generalization.

---

2. Data Augmentation

Artificially create new training samples.

Common image augmentations:

- Rotation
- Flipping
- Cropping
- Zooming
- Brightness adjustment
- Noise addition

Example:

Original image



Augmented images



 (flipped)
 (zoomed)

Benefits:

- Larger effective dataset
- Better robustness
- Reduced overfitting

---

3. Simpler Model

Reduce model complexity.

Methods:

- Fewer layers
- Fewer neurons
- Lower polynomial degree
- Simpler algorithms

Example:

Instead of a 100-layer neural network, use a 10-layer network if appropriate.

---

4. Feature Selection

Remove unnecessary or irrelevant features.

Example:

Predicting house prices:

Useful:

- Area
- Bedrooms

- Location

Not useful:

- House owner's favorite color
- Owner's phone number

Removing irrelevant features reduces overfitting.

---

5. Regularization

Regularization adds a penalty to the loss function to discourage overly complex models.

L1 Regularization (Lasso)

Penalty:

$$\text{Loss} = \text{Original Loss} + \lambda \sum |w|$$

Characteristics:

- Produces sparse models
- Some weights become exactly zero
- Useful for feature selection

L2 Regularization (Ridge)

Penalty:

$$\text{Loss} = \text{Original Loss} + \lambda \sum w2$$

Characteristics:

- Shrinks weights toward zero
- Rarely makes them exactly zero
- Most commonly used in neural networks

Elastic Net

Combines L1 and L2 regularization.

---

6. Dropout (Neural Networks)

Dropout randomly disables a fraction of neurons during training.

Example:

Before

O O O O O O

After Dropout (50%)

O X O X O X

X = temporarily disabled

Benefits:

- Prevents neurons from relying too heavily on one another
- Improves generalization
- Common dropout rates: 0.2–0.5

---

7. Early Stopping

Stop training when validation performance stops improving.

Example:

Epoch	Training Loss	Validation Loss
1	0.80	0.85
5	0.35	0.40
10	0.15	0.25
15	0.05	0.27

Training loss keeps decreasing, but validation loss starts increasing after epoch 10. Early stopping would stop training around epoch 10.

---

8. Noise Reduction

Improve data quality by:

- Removing duplicate records
- Correcting incorrect labels
- Handling missing values
- Filtering out outliers when appropriate

Cleaner data helps the model focus on meaningful patterns.

---

9 Hyperparameter Tuning

Adjust parameters such as:

- Learning rate
- Tree depth
- Number of layers
- Number of neurons
- Batch size
- Regularization strength (λ)
- Dropout rate

Proper tuning can improve generalization and reduce overfitting.

Apply Regularization

• Regularization is a technique that helps reduce overfitting or reduce variance in a Neural network by penalizing for complexity. The idea is that certain complexities in our model may make our model unlikely to generalize well, even though the model fits the training data.

• To implement regularization is to simply add a term to our loss function that penalizes for large weights.

• Y= x1w1 +x2w2+ ... +xnwn

• Y = 15+x1*20 +x2*1.5+ ... +xn*30

• Y = x1*2 +x2*0.2+ ... +xn*5

L1 regularization

• L1 regularization, also known as L1 norm or Lasso (in regression problems), combats overfitting by shrinking the parameters towards 0. This makes some features obsolete.
• It's a form of feature selection, because it shrinks the less important feature's coefficient to zero.
• Mathematically, L1 regularization is expressed by extending loss function as Below:

$$\text{Loss}=\text{Original Loss}+\lambda\sum|w|$$

• L1 regularization, we are penalizing the absolute value of the weights.

L2 regularization

• L2 regularization, or the L2 norm, or Ridge (in regression problems), combats
overfitting by forcing weights to be small, but not making them exactly 0.
• The regularization term that we add to the loss function when performing L2 regularization is the sum of squares of all of the feature weights:

$$\textbf{Loss}=\textbf{Original Loss}+\lambda\sum\textbf{w2}$$

The term λ is called the regularization parameter, and this is another hyperparameter that we'll have to choose and then test and tune in order to choose the correct number for our specific model.

The **Original Loss** (also called the **cost function** or **objective function**) measures how far the model's predictions are from the actual values. The formula depends on the type of machine learning problem.

1. Mean Squared Error (MSE) – Regression

Used for predicting continuous values (e.g., house prices).

$MSE = n1/ni = 1 \sum n(yi - y\hat{}i)^2$

Where:

- n = number of training examples
- yi = actual value
- y^i = predicted value

Actual values: [10,20,30]

Predicted values: [12,18,29]

$MSE = (10-12)^2 + (20-18)^2 + (30-29)^2/3 = 4+4+1=3$

Cross-Entropy Loss – Classification

Used for classification problems.

Binary Cross-Entropy

- yi = actual class (0 or 1)
- y^i = predicted probability